

Analyzing Traffic Flow: Using Steady States and Markov Chains to Predict the Busiest CT Routes and Individual Traffic Paths

By Kyle Levesque, Eashan Hatti, and Wyatt Lake

Introduction

In this project, the following questions were analyzed:

1. Given traffic flow data in Connecticut, what are the routes where cars are most likely to be in the long run or in other words, what are the busiest routes?
2. Given a starting place, where is the car likely to end up after several iterations?

Data Collection

We first had to collect location data on all the roads in Connecticut. After trying a few different sources, we settled on collecting from the CTDOT Open Data website in the form of shapefiles, which is an open data format for storing geographic data. From there Python and the libraries Pandas, Geopandas, and Shapely were used to transform the shapefile vector format into a CSV format storing the names and coordinates (in longitude/latitude) of each road.

Linear Algebra Tools

In this project, we utilized Markov chains and steady states to analyze our data. A Markov chain is a system that describes a sequence of possible outcomes. In a Markov chain, the state to which it will go next depends only on its present state and does not depend on the earlier history of the system. The state that the system will occupy at any stage is determined only in terms of probabilities, and if a Markov chain is in a particular state, then the probabilities that it goes to various states at the next stage of its evolution are called transition probabilities. Using these transition probabilities, a transition matrix P can be created which is a square matrix where the columns represent these transition probabilities and sum to 1. Given an initial state vector s_0 , the next state vector can be found by multiplying the state vector by the transition matrix: $s_1 = Ps_0$. The next state vector after that can be found in a similar manner: $s_2 = Ps_1$. This process was utilized to find the possible routes that a car may end up on and the probability of being on each route given a starting point.

A steady state vector represents the long-term probabilities of a system being in each state after many iterations of a transition matrix. Essentially, it means that the system has reached equilibrium and each state's probability stays constant. Therefore, a steady state vector is a probability vector that does not change after applying the transition matrix and is a vector s such that $Ps = s$. One way to find this steady-state vector is to find the eigenvector of the transition matrix with an eigenvalue of 1 because then $Ps = 1*s$, which is the definition of a steady-state vector. This is the approach that was used in this project.

Analysis

The analysis for this project was done in R. Given the traffic flow data collected, the first task was to transform this data into a transition matrix giving the probability of going from each unique node to another. The traffic flow data included the average daily traffic flow along

different routes with starting and ending coordinates of this flow. Therefore, using analysis in R, two new statistics were calculated: the total amount of traffic flow from one specific location to another and the total amount of traffic flow from one specific location to any location. From these statistics, the probability of going from one location to another location was found by dividing the total amount of traffic flow from one specific location to another specific location by the total amount of traffic flow from the first specific location to any location.

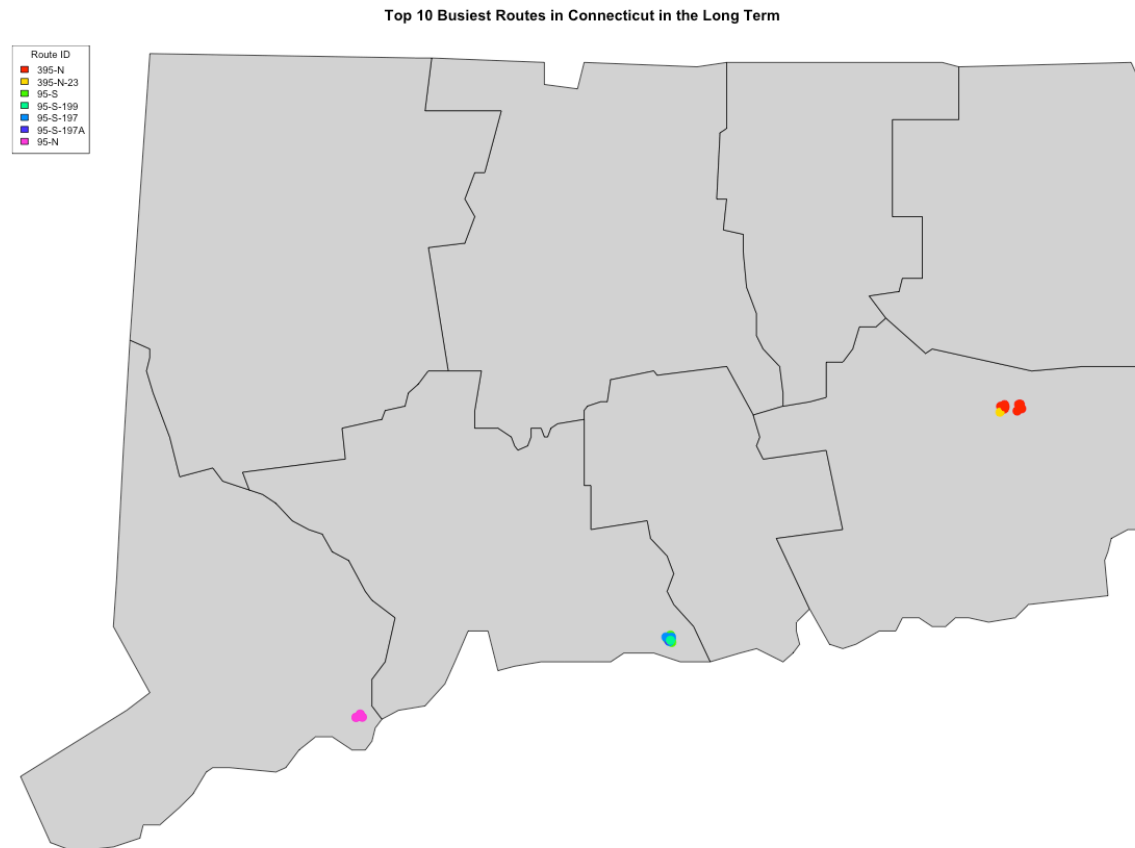
Now that we had turning probabilities from one location to another, we could create a transition matrix. First, every unique node (or location) was found and these become the row names and the column names. Then, using a for loop in R, the transition probabilities were filled in for this matrix. To make coding simpler, the transition probabilities were filled in by row, so each row summed up to 1 instead of the columns, but this did not change the use of the matrix. However, one problem that did arise was that some locations were dead ends, that is, they did not lead to another location. Therefore, many rows did not add up to 1. Out of 5333 locations about 1000 of them were dead ends. To combat this, an additional row and column was added, and any rows that were a dead end gained a 1 in this final column to signify that it was a dead end and to ensure all row sums were 1. Doing this ensured that we would be able to use the transition matrix for our given purposes.

The next step in our process was to find the steady state vector of this transition matrix. Thankfully, due to the large size of the transition matrix at 5334 x 5334, R had functions to find eigenvectors. These functions were utilized to find an eigenvector associated with an eigenvalue of 1. This eigenvector corresponded to the steady-state vector due to the reasons described in Linear Algebra Tools. A dataframe was then created that had each node (location) with its associated steady state probability. As expected, the sum of the steady state probabilities were 1. Here is a snippet of what the dataframe looked like.

	node	steady_state_prob
-		
1	71.79337842,41.79450363	-1.16E-16
-		
2	71.79555225,41.79281112	-6.79E-17
-		
3	71.79618873,41.79327665	-4.80E-17
-		
4	71.79781174,41.44143232	-7.71E-16
-		1.18653976443356e-
5	71.79788921,41.44169654	16

The 10 highest steady state probabilities were then found, with their corresponding location. These locations represented the locations where cars were most likely to end up in the long term, an answer to our first question. The route ID for each location was then found and plotted onto a map of Connecticut.

See below



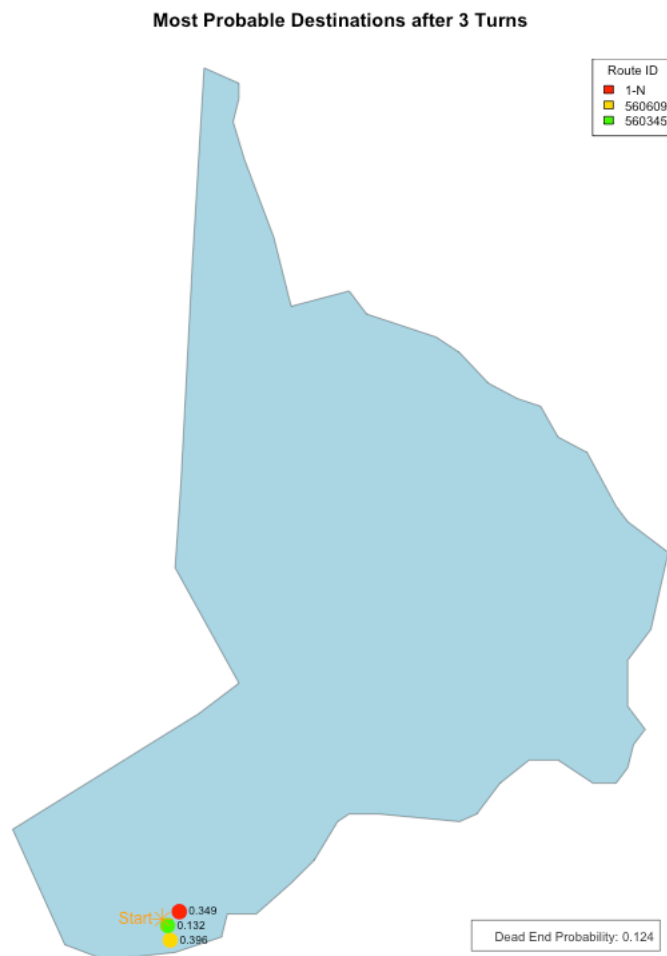
To answer our second question, we selected a random location to start at using R but ensured that it was not a dead end. Otherwise, it would lead nowhere. Then, we created an initial state vector that had all 0's except for the location that we chose, which had a value of 1. 3 iterations were chosen. A loop was then used to calculate the third state vector. This loop started with the initial state vector and multiplied it by the transition matrix to find the first state vector, then multiplied the first state vector by the transition matrix to find the second state vector, and so on until the third state vector was found. The non-zero probabilities from this vector were then extracted and put into a dataframe.

See below

node	probability
<dbl>	<dbl>
-73.56685598,41.02711768	0.3957597
-73.5578955,41.04838824	0.3490140
-73.56944807,41.03807148	0.1315514
dead_end	0.1236749

Therefore, for this given starting point, there are four different possible outcomes after 3 iterations. It can either end up at 3 other locations or end up at a dead end. This can also be

mapped out. The starting point was in Fairfield County, Connecticut, and thus below is a map of Fairfield County with the starting point and possible outcomes. This analysis answered our section question that we aimed to answer.



Appendix: R-code

This is the complete R-code that was used to analyze the data.

```
title: "Math_222_Final_Project"
output: html_document
date: "2025-04-08"
---

```{r setup, include=FALSE}
knitr::opts_chunk$set(echo = TRUE)
```

```{r}
#Reading in the data
traffic2023 <- read.csv("/Users/kylelevesque/Downloads/data 2/2023.csv")
head(traffic2023)
```

```

dim(traffic2023)
summary(traffic2023$aadt)

'''

''' {r}
#Just getting a feel for the data
length(unique(traffic2023$route_id))
sum(duplicated(traffic2023$route_id))
#There are some duplicated routes so can't really so can't really analyze by route_id but instead coordinates

'''

''' {r}
#This code just gets the latitude and longitude into coordinates
traffic2023$from <- paste(traffic2023$slat, traffic2023$slon, sep = ",")
traffic2023$to <- paste(traffic2023$elat, traffic2023$elon, sep = ",")

#This gets the amount of traffic flow from one specific location to another specific location
traffic_flow <- aggregate(aadt ~ from + to, data = traffic2023, sum)

#This is the total amount of traffic flow from one specific location to any location
total_outbound <- aggregate(aadt ~ from, data = traffic_flow, sum)

#This merges the total_outbound data with the traffic_flow data
traffic_flow <- merge(traffic_flow, total_outbound, by = "from", suffixes = c("_to", "_total"))

traffic_flow$turn_prob <- traffic_flow$aadt_to / traffic_flow$aadt_total

Step 5: View results
head(traffic_flow)
'''

''' {r}
#Get all unique "nodes". This is takes all of the possible coordinate combinations from the from and to columns and gets the unique locations.
nodes <- unique(c(traffic_flow$from, traffic_flow$to))
length(nodes)
dim(traffic_flow)
#This creates an empty stochastic matrix with the nodes
n <- length(nodes)
P <- matrix(0, nrow = n, ncol = n)
rownames(P) <- nodes
colnames(P) <- nodes
#This fills in the matrix with the corresponding turn probability data
for (i in 1:nrow(traffic_flow)) {
 from_node <- traffic_flow$from[i]
 to_node <- traffic_flow$to[i]
 prob <- traffic_flow$turn_prob[i]
 P[from_node, to_node] <- prob
}

#This gets the row sums to make sure that they add up to 1
row_sums <- rowSums(P)
summary(row_sums)

#There appears to be some dead ends as the mean across rows is not 1. Therefore, there must be some locations that do not go anywhere.
dead_ends <- rownames(P)[rowSums(P) == 0]
length(dead_ends)
#There appears to be 1090 dead ends out of 5333 intersections, which could be a little high
#write.csv(P, file = "/Users/kylelevesque/Downloads/stochastic_matrix.csv", row.names = TRUE)
'''

''' {r}
dead_end_flag <- as.numeric(rowSums(P) == 0)
P_with_col <- cbind(P, is_dead_end = dead_end_flag)
dim(P_with_col)
new_row <- rep(0, ncol(P_with_col))
P_augmented <- rbind(P_with_col, new_row)

```

```

rownames(P_augmented)[nrow(P_augmented)] <- "dead_end"
P_augmented[nrow(P_augmented), ncol(P_augmented)] <- 1
summary(rowSums(P_augmented))
dim(P_augmented)
#write.csv(P_augmented, file = "/Users/kylelevesque/Downloads/MATH 222/stochastic_matrix2.csv", row.names = T)
...

```{r}
#Markov Chain/Steady State, Code Finds Eigenvector with Eigenvalue of 1

#computes eigenvalues and eigenvectors
eig <- eigen(t(P_augmented))

#Finds index of the eigenvalue with largest real part, which is 1 for a stochastic matrix.
idx <- which.max(Re(eig$values))

#Extracts the real part of the eigenvector corresponding to eigenvalue 1
steady_state <- Re(eig$vectors[, idx])

#displays steady state vector
head(steady_state)

#verifies steady state vector sums to 1
sum(steady_state)

...

```{r}
#Creates a dataframe with steady state vector probabilities and nodes
node <- rownames(P_augmented)
steady_state_df <- data.frame(
 node = node,
 steady_state_prob = steady_state
)
#write.csv(steady_state_df, file = "/Users/kylelevesque/Downloads/MATH 222/steady_state_df.csv", row.names = T)
steady_state_df <- steady_state_df[order(-steady_state_df$steady_state),]
steady_state_df$lon <- as.numeric(sub(".", "*", "", steady_state_df$node))
steady_state_df$lat <- as.numeric(sub(".", "*", "", steady_state_df$node))

...

```{r}
library(maps)
library(mapproj)
...

```{r}
library(dplyr)
node_route_df <- traffic2023 %>%
 select(route_id, from, to) %>%
 tidyr::pivot_longer(cols = c(from, to), names_to = "type", values_to = "node") %>%
 distinct()
node_route_df
...

```{r}
steady_state_with_routes <- merge(steady_state_df, node_route_df, by = "node", all.x = TRUE)
top10_with_routes <- merge(steady_state_df[1:10, ], steady_state_with_routes[, c("node", "route_id")], by = "node")
top10_with_routes_unique <- top10_with_routes[!duplicated(top10_with_routes$node), ]
...

```{r}
png("CT_busiest_intersections.png", width = 1200, height = 900)

map("county", "connecticut", col = "lightgray", fill = TRUE, bg = "white")
title("Top 10 Busiest Routes in Connecticut in the Long Term")

```

```

Create a color palette for route_ids
Generate a unique color for each route_id
unique_route_ids <- unique(top10_with_routes$route_id)
colors <- rainbow(length(unique_route_ids))

Assign colors to the routes in your dataframe
top10_with_routes$color <- colors[match(top10_with_routes$route_id, unique_route_ids)]

Plot the top 10 busiest intersections with the assigned colors
points(jitter(top10_with_routes$lon, amount = 0.005), jitter(top10_with_routes$lat, amount = 0.005), col = top10_with_routes$color, pch = 19,
cex = 1.5)

Add a legend with route_id and the corresponding color
legend("topleft",
 legend = unique_route_ids,
 fill = colors,
 title = "Route ID",
 cex = 0.8)

dev.off()
```



```

```{r}
#Start at a random road and see probability of ending up at different nodes
dead_ends <- rowSums(P) == 0
valid_nodes <- rownames(P)[!dead_ends]

set.seed(500) # for reproducibility
start_node <- sample(valid_nodes, 1)
start_index <- which(rownames(P_augmented) == start_node)
```

```{r}
v <- rep(0, nrow(P_augmented))
v[start_index] <- 1
```

```{r}
steps <- 3
v_current <- v

# %*% is operator for matrix multiplication
for (i in 1:steps) {
  v_current <- v_current %*% P_augmented
}
```

```{r}
results <- data.frame(
  node = rownames(P_augmented),
  probability = as.vector(v_current)
)

results_sorted <- results[order(-results$probability), ]
random_prob_results <- results_sorted[results_sorted$probability != 0, ]

random_prob_results$lon <- as.numeric(sub(".", "*", "", random_prob_results$node))
random_prob_results$lat <- as.numeric(sub(".", "*", "", random_prob_results$node))
```

```{r}
from_matches <- merge(random_prob_results, traffic2023[, c("route_id", "from")],
  by.x = "node", by.y = "from", all.x = TRUE)

# Match to_node
to_matches <- merge(random_prob_results, traffic2023[, c("route_id", "to")],
  by.x = "node", by.y = "to", all.x = TRUE)

# Combine both matches
combined_matches <- rbind(from_matches, to_matches)

```


```

```

Remove duplicates (some nodes may appear in both from and to)
random_path <- combined_matches[c(1, 2, 7, 8),]

#random_path <- write.csv(random_path, file = "/Users/kylelevesque/Downloads/MATH 222/random_path.csv", row.names = T)

...

```{r}
unique_route <- unique(random_path$route_id[1:3])
colors <- rainbow(length(unique_route_ids))

random_path$color <- colors[match(random_path$route_id, unique_route)]

start_lon <- as.numeric(sub(".", "*", "", start_node))
start_lat <- as.numeric(sub(".", "*", "", start_node))
...

```{r}
png("MostProbableDestinations.png", width = 600, height = 800)
library(maps)

map("county", "connecticut,fairfield", col = "lightblue", fill = TRUE, lwd = 0.5)
title("Most Probable Destinations after 3 Turns")

points(random_path$lon[1:3], random_path$lat[1:3], pch = 19, col = random_path$color, cex = 2)
text(random_path$lon, random_path$lat,
 labels = round(random_path$probability, 3), pos = 4, cex = 0.7, col = "black")

points(start_lon, start_lat, pch = 8, col = "orange", cex = 2)
text(start_lon, start_lat, labels = "Start", pos = 2, cex = 1, col = "orange")

legend("topright",
 legend = unique_route,
 fill = colors,
 title = "Route ID",
 cex = 0.8)

legend("bottomright",
 legend = paste("Dead End Probability:", round(random_path$probability[4], 3)),
 bty = "o",
 cex = 0.8,
 text.col = "gray20",
 box.col = "gray40",
 bg = "white",
 inset = c(0.02, 0.02)
)

dev.off()
...

```